

## Лабораторна робота 5. Масштабування в Kubernetes

**Мета:** Освоїти механізми горизонтального масштабування в Kubernetes

### Передумови

Перед початком роботи переконайтесь, що на вашій машині встановлено:

- Виконана лабораторна робота 4 (Система розгорнута в Kubernetes, LGTM стек працює)

### Система

Ви працюватимете з готовим 3-рівневим веб-застосунком (todo-app), який складається з трьох компонентів:

- Frontend: React-застосунок, інтерфейс користувача.
- Backend: Python-сервер (FastAPI), який обробляє запити та працює з даними.
- База даних: PostgreSQL.

Репозиторій: [https://github.com/igoris/cloud\\_computing\\_lab1](https://github.com/igoris/cloud_computing_lab1)

Система моніторингу буде складатись з наступних компонентів:

- Prometheus – збирає метрики (числові показники: CPU, RAM, кількість Pods).
- Loki – зберігає та індексує логи контейнерів.
- Grafana Alloy – агент, який читає логи з вузлів і надсилає їх у Loki. Замінює Promtail, який deprecated з 2026 року.
- Grafana – веб-інтерфейс для дашбордів, графіків та перегляду логів.

Разом із Tempo (трейси) та Mimir (масштабоване сховище метрик) ця комбінація відома як LGTM-стек від Grafana Labs. Mimir тут заміщений Prometheus (він легший для Minikube, а знання PromQL повністю переноситься на Mimir).

### Завдання 1. Resource requests, limits і перевірка метрик

Зверніть увагу, якщо в команді зустрічаються назви, написані курсивом, потрібно їх змінити на власні.

Активуйте metrics-server. Він потрібен для команд `kubectl top` і для роботи НРА:

```
minikube addons enable metrics-server
```

Зачекайте 1–2 хвилини і перевірте, що метрики доступні:

```
kubectl top nodes
```

```
kubectl top pods
```

Додайте в звіт поточне використання CPU і пам'яті для frontend, backend, postgres подів.

Додайте секцію resources до контейнерів у Deployment-маніфестах frontend, backend і postgres.

Можна використати такі початкові значення:

- Requests:
  - Cpu: 100m
  - Memory: 128Mi
- Limits:
  - Cpu: 500m
  - Memory: 256Mi

Для БД варто відразу задати більші значення для пам'яті (256Mi/512Mi).

Застосуйте маніфести, перевірте, що поди перезапустились. Ще раз перевірте використання ресурсів. Додайте результат в звіт.

Тимчасово змініть ресурси backend-контейнера на дуже малі значення:

- Requests:
  - Cpu: 50m
  - Memory: 16Mi
- Limits:
  - Cpu: 200m
  - Memory: 16Mi

Відправте декілька запитів на бекенд. Що відбувається з подом?

Після експерименту поверніть нормальні значення ресурсів backend-контейнера.

Додайте файли в репозиторій і додайте в звіт посилання на коміт, бранч чи пулл реквест з цими файлами.

Дайте відповіді на питання:

1. Чим відрізняються requests та limits?
2. Що таке OOMKilled?
3. Чому requests.memory не може бути більшим за limits.memory?

## **Завдання 2. Ручне масштабування і налаштування НРА**

Виконайте ручне масштабування backend:

```
kubectl scale deployment todo-backend --replicas=3
```

Додайте в звіт результати виконання команд:

```
kubectl get pods -l app=todo-backend
```

```
kubectl get endpoints todo-backend
```

Поясніть, що ці результати показують.

Поверніть бекенд до 1 репліки, перевірте, що зайві поди завершилися.

Створіть маніфест для HorizontalPodAutoscaler для бекенду, встановіть minReplicas: 1, maxReplicas: 5. Вкажіть метрику для НРА: cpu, target: Utilization, 50. Застосуйте маніфест.

Перевірте НРА:

```
kubectl get hpa
```

Додайте результат в звіт.

Дайте відповіді на питання:

1. Що змінилось у endpoints сервісу після збільшення кількості podів?
2. Чому НРА потребує requests.cpu?
3. Чим НРА відрізняється від VPA?

## **Завдання 3. Навантажувальне тестування**

Підготуйте три термінали.

У першому терміналі спостерігайте за НРА

```
kubectl get hpa todo-backend -w
```

У другому терміналі спостерігайте за backend pod-ами:

```
kubectl get pods -l app=todo-backend -w
```

У третьому терміналі запусить навантаження всередині Kubernetes-кластера:

```
kubect1 run loadtest --rm -it --image=busybox -- sh -c "while true; do wget -q -O- http://service_name:port/api/todos; done"
```

Додайте в звіт таймлайн за прикладом:

| Час | Подія                       |
|-----|-----------------------------|
| 0с  | старт навантаження          |
| 5с  | CPU > 50%                   |
| 6с  | HPA змінив desired replicas |
| 10с | нові podи стали Ready       |

...

Якщо запити не генерують достатньо навантаження, тимчасово зменшіть виділені ресурси бекенд-контейнера та повторіть тест.

В Grafana побудуйте графік CPU backend podів. Перезапустіть тест, додайте в звіт скріншот графіка.

Зупиніть тест, додайте в звіт результати спостереження за scale-down (він як правило займає більше часу, ніж scale-up).

Дайте відповіді на питання:

1. Чому backend можна масштабувати до кількох реплік?
2. Які можуть виникнути проблеми при додаванні реплік для postgres?
3. Чим ручне масштабування відрізняється від HPA?
4. Чому HPA масштабує вниз повільніше, ніж вгору?

## Список літератури:

1. <https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource>
2. <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
3. <https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/>